

Pre-Liminary Training Tasks

1. Docker-based simulated laboratory

You need to create a self-contained Docker laboratory representing several IoT devices.

Required simulated devices

You should create at least three logical IoT device profiles.

Device A: Insecure smart camera

Simulated features:

- a. HTTP management interface
- b. Default credentials
- c. Telnet service
- d. Unencrypted MQTT
- e. Hard-coded API key
- f. Outdated software component
- g. Weak or missing security logging
- h. Privacy document with missing retention information

Device B: Partially hardened smart camera

Simulated features:

- a. Telnet removed
- b. Default password changed
- c. HTTPS enabled with weak certificate configuration
- d. MQTT still unencrypted
- e. Some logging enabled
- f. Update process present but unsigned
- g. Incomplete privacy documentation

Device C: Hardened smart camera

Simulated features:

- a. HTTPS only
- b. Strong credentials
- c. MQTT over TLS
- d. No unnecessary services
- e. Signed simulated firmware package
- f. Security logging enabled
- g. Updated components
- h. Complete vendor documentation
- i. Clear retention and deletion evidence

The same application may be configured differently through Docker Compose profiles, environment variables and mounted configuration files.

2. Required Docker laboratory architecture

The Docker Compose environment should include:

auditor-web
auditor-api
auditor-worker
auditor-database
device-insecure
device-partial
device-hardened
mqtt-broker-insecure
mqtt-broker-secure
traffic-capture
document-store

Optional supporting containers may include:

- Vulnerability database mirror
- Mock update server
- Reverse proxy
- Log collector
- Test certificate authority

Network design

Create at least two Docker networks:

Audit network

Used by the auditor to reach the simulated devices.

Internal service network

Used for backend services that should not be exposed to the simulated devices.

You should document:

- a. Container names
- b. IP ranges
- c. Exposed ports
- d. Trust boundaries
- e. Data flows
- f. Which services should be reachable
- g. Which services should remain isolated

Three-day training sprint

Day 1: Docker IoT laboratory and network-security basics

Build and understand a safe simulated IoT environment.

Tasks

1. Simulated IoT service

Create an intentionally insecure Flask or FastAPI service representing a smart camera.

It must include:

- Login page
- Device information endpoint
- Configuration endpoint
- Firmware version endpoint
- Default username and password
- Plain HTTP
- At least one administrative endpoint

2. Network services

Add simulated IoT network services:

- Telnet-like service
- MQTT broker
- HTTP service
- Optional SSH service

The services must be reachable only inside the Docker laboratory.

3. Docker infrastructure

Create:

- Dockerfiles
- Docker Compose file
- Audit network
- Internal network
- Volumes
- Environment configuration
- Health checks

4. Threat model and evidence model

Produce:

- Architecture diagram
- Trust-boundary diagram
- Basic STRIDE-style threat model
- Initial JSON evidence schema

Required team output

By the end of Day 1:

- A working Docker Compose environment
- One simulated IoT device
- At least three exposed services
- A network diagram
- A threat model
- A device inventory
- A README explaining how to start and stop the laboratory

Acceptance test

You must be able to run docker and then:

- Reach the device web interface
 - Connect to the MQTT service
 - Detect at least three open ports
 - View the device's simulated metadata
-

Day 2: Manual cybersecurity assessment

The focus here is to collect evidence before automating the process.

Tasks

1. Web and authentication assessment

Test:

- Default credentials
- Anonymous access
- Weak session behaviour
- Missing security headers
- Unprotected administrative endpoints

2. Network and protocol assessment

Run:

- Nmap service detection
- HTTP inspection
- MQTT testing
- TLS testing where applicable
- Packet capture

3. Simulated firmware analysis

Create a sample firmware archive containing:

- Version file
- Configuration files
- Hard-coded password
- API key
- Certificate or key file
- Package manifest
- Update script

Then analyse it using:

- file
- strings
- grep
- YARA
- Syft
- Gripe or an equivalent vulnerability scanner

4. Evidence normalisation

For each observed issue, create a structured evidence record containing:

- Evidence ID
- Device ID
- Tool
- Tool version
- Command
- Timestamp
- Finding
- Raw output location
- Confidence
- Hash of evidence file

Required team output

At least eight manual findings, including:

- Default credentials
- Exposed insecure service
- Unencrypted protocol

- Hard-coded secret
- Outdated package
- Weak TLS or missing TLS
- Missing logging
- Missing privacy or update evidence

Acceptance test

For each finding, you must show:

Raw tool output

- structured evidence
 - security interpretation
 - suggested remediation
-

Day 3: Saudi policy mapping and policy-as-code

Convert written cybersecurity requirements into machine-readable controls.

Tasks

1. Saudi control mapping

Select and document the first five controls from relevant Saudi sources.

The initial controls should cover:

1. Device identification
2. Default credentials
3. Unnecessary services
4. Insecure protocols
5. TLS or secure communications

2. Test-to-control mapping

For every control, define:

- Which Docker service creates the evidence
- Which command or tool performs the test
- Which result indicates Pass
- Which result indicates Fail
- Which result indicates Inconclusive

3. YAML policy format

Implement the initial control schema in YAML.

Every control must contain:

- Control ID
- Title
- Saudi source mapping
- Applicability conditions
- Required evidence
- Automated test IDs
- Pass conditions
- Fail conditions
- Partial conditions
- Inconclusive conditions
- Severity
- Remediation

4. Minimal policy engine

Create a Python module that:

- Loads one YAML control
- Reads one evidence JSON object
- Applies simple verdict logic
- Outputs a verdict JSON object

Required team output

A working demonstration:

Simulated device

- network test
- evidence JSON
- YAML policy
- verdict JSON

Acceptance test

At least two controls must generate correct Pass and Fail verdicts for different simulated configurations.